

Software Engineering – Unit 1

We will cover all the topics for BTECH and MCA.

Unit-1 has exactly the same topics for BTECH and MCA.

Introduction to Software Engineering, Software Components, Software Characteristics, Software Crisis, Software Engineering Processes, Similarity and Differences from Conventional Engineering Processes, Software Quality Attributes. Software Development Life Cycle (SDLC) Models: Water Fall Model, Prototype Model, Spiral Model, Evolutionary Development Models, Iterative Enhancement Models.

Introduction to Software Engineering

Software is a program, or collection of programs, that contains instructions to perform specific functions. Engineering is the process of creating something with a specific aim in mind and figuring out how to do it efficiently.

Software Engineering is the process of designing, developing, testing, and maintaining software. It is a methodical and disciplined approach to software development that strives to produce high-quality, dependable, and maintainable software. Software engineering encompasses a wide range of techniques, tools, and procedures such as requirements analysis, design, testing, and maintenance.

Software Components

Program –

A computer program is a list of instructions that tell a computer what to do.

Documentation –

Source information about the product contained in design documents, detailed code comments, etc.

Operating Procedures –

Set of step-by-step instructions compiled by an organization to help workers carry out complex routine operations.

Code: the instructions that a computer executes in order to perform a specific task or set of tasks.

Data: the information that the software uses or manipulates.

User interface: the means by which the user interacts with the software, such as buttons, menus, and text fields.

Libraries: pre-written code that can be reused by the software to perform common tasks.

Documentation: information that explains how to use and maintain the software, such as user manuals and technical guides.

Test cases: a set of inputs, execution conditions, and expected outputs that are used to test the software for correctness and reliability.

Configuration files: files that contain settings and parameters that are used to configure the software to run in a specific environment.

Build and deployment scripts: scripts or tools that are used to build, package, and deploy the software to different environments.

Metadata: information about the software, such as version numbers, authors, and copyright information.

Software Characteristics

The characteristics of the software include:

- It is intangible, meaning it cannot be touched.
- It is non-perishable, meaning it does not degrade over time.
- It is easy to replicate, meaning it can be copied and distributed easily.
- It can be complex, meaning it can have many interrelated parts and features.
- It can be difficult to understand and modify, especially for large and complex systems.
- It can be affected by changing requirements, meaning it may need to be updated or modified as the needs of users change.
- It can be affected by bugs and other issues, meaning it may need to be tested and debugged to ensure it works as intended.

Software Characteristics contd....

Components of Software Characteristics:

Functionality - It refers to the degree of performance of the software against its intended purpose.

Functionality refers to the set of features and capabilities that a software program or system provides to its users. It is one of the most important characteristics of software, as it determines the usefulness of the software for the intended purpose. Examples of functionality in software include:

- Data storage and retrieval
- Data processing and manipulation
- User interface and navigation
- Communication and networking
- Security and access control
- Reporting and visualization
- Automation and scripting

The more functionality a software has, the more powerful and versatile it is, but also the more complex it can be. It is important to balance the need for functionality with the need for ease of use, maintainability, and scalability.



Software Characteristics contd....

Reliability

A set of attributes that bears on the capability of software to maintain its level of performance under the given condition for a stated period of time.

Reliability is a characteristic of software that refers to its ability to perform its intended functions correctly and consistently over time. Reliability is an important aspect of software quality, as it helps ensure that the software will work correctly and not fail unexpectedly.

Examples of factors that can affect the reliability of software include:

1. Bugs and errors in the code
2. Lack of testing and validation
3. Poorly designed algorithms and data structures
4. Inadequate error handling and recovery
5. Incompatibilities with other software or hardware

To improve the reliability of software, various techniques, and methodologies can be used, such as testing and validation, formal verification, and fault tolerance.

Software is considered reliable when the probability of it failing is low and it is able to recover from the failure quickly, if any.

Software Characteristics contd....

Efficiency

It refers to the ability of the software to use system resources in the most effective and efficient manner. The software should make effective use of storage space and executive command as per desired timing requirements.

Efficiency is a characteristic of software that refers to its ability to use resources such as memory, processing power, and network bandwidth in an optimal way. High efficiency means that a software program can perform its intended functions quickly and with minimal use of resources, while low efficiency means that a software program may be slow or consume excessive resources.

Examples of factors that can affect the efficiency of the software include:

1. Poorly designed algorithms and data structures
2. Inefficient use of memory and processing power
3. High network latency or bandwidth usage
4. Unnecessary processing or computation
5. Unoptimized code

To improve the efficiency of software, various techniques, and methodologies can be used, such as performance analysis, optimization, and profiling.

Efficiency is important in software systems that are resource-constrained, high-performance, and real-time systems. It is also important in systems that need to handle a large number of users or transactions simultaneously.

Software Characteristics contd....

Usability

It refers to the extent to which the software can be used with ease. the amount of effort or time required to learn how to use the software.

Maintainability

It refers to the ease with which modifications can be made in a software system to extend its functionality, improve its performance, or correct errors.

Portability

A set of attributes that bears on the ability of software to be transferred from one environment to another, without minimum changes.

Software Crisis

Software Crisis is a term used in computer science for the difficulty of writing useful and efficient computer programs in the required time. The software crisis was due to using the same workforce, same methods, and same tools even though rapidly increasing software demand, the complexity of software, and software challenges. With the increase in software complexity, many software problems arise because existing methods were insufficient.

If we will use the same workforce, same methods, and same tools after the fast increase in software demand, software complexity, and software challenges, then there arise some problems like software budget problems, software efficiency problems, software quality problems, software management, and delivery problem, etc. This condition is called a Software Crisis.

Causes of Software Crisis

Causes of Software Crisis

1. The cost of owning and maintaining software was as expensive as developing the software.
2. At that time Projects were running overtime.
3. At that time Software was very inefficient.
4. The quality of the software was low quality.
5. Software often did not meet user requirements.
6. The average software project overshoots its schedule by half.
7. At that time Software was never delivered.
8. Non-optimal resource utilization.
9. Challenging to alter, debug, and enhance.
10. The software complexity is harder to change.

Causes of Software Crisis

Factors Contributing to Software Crisis

1. Poor project management.
2. Lack of adequate training in software engineering.
3. Less skilled project members.
4. Low productivity improvements.

Solution of Software Crisis

There is no single solution to the crisis. One possible solution to a software crisis is Software Engineering because software engineering is a systematic, disciplined, and quantifiable approach. For preventing software crises, there are some guidelines:

1. Reduction in software over budget.
2. The quality of the software must be high.
3. Less time is needed for a software project.
4. Experienced and skilled people working on the software project.
5. Software must be delivered.
6. Software must meet user requirements.

Software Engineering Processes

A software engineering process is the model chosen for managing the creation of software from initial customer inception to the release of the finished product.

The chosen process usually involves techniques such as:

- Analysis,
- Design,
- Coding,
- Testing and
- Maintenance

Similarity and Differences from Conventional Engineering Processes

Software Engineering Process

Software Engineering Process is an engineering process that is mainly related to computers and programming and developing different kinds of applications through the use of information technology.

Conventional Engineering Process

Conventional Engineering Process is an engineering process that is highly based on empirical knowledge and is about building cars, machines, and hardware. It is a process that mainly involves science, mathematics, etc.

Similarity and Differences from Conventional Engineering Processes

Software Engineering Process

Software Engineering Process is an engineering process that is mainly related to computers and programming and developing different kinds of applications through the use of information technology.

Conventional Engineering Process

Conventional Engineering Process is an engineering process that is highly based on empirical knowledge and is about building cars, machines, and hardware. It is a process that mainly involves science, mathematics, etc.

Similarity and Differences from Conventional Engineering Processes

Similarities Between Software Engineering Process and Conventional Engineering Process

- Both Software Engineering and Conventional Engineering Processes become automated after some time.
- Both these processes are making our day-to-day place better.
- Both these processes have a fixed working time.
- Both processes must consist of deeper knowledge.

Similarity and Differences from Conventional Engineering Processes

Similarities Between Software Engineering Process and Conventional Engineering Process

- Both Software Engineering and Conventional Engineering Processes become automated after some time.
- Both these processes are making our day-to-day place better.
- Both these processes have a fixed working time.
- Both processes must consist of deeper knowledge.

Similarity and Differences from Conventional Engineering Processes

Software Engineering Process	Conventional Engineering Process
Software Engineering Process is a process that majorly involves computer science, information technology, and discrete mathematics.	The conventional Engineering Process is a process that majorly involves science, mathematics, and empirical knowledge.
It is mainly related to computers, programming, and writing codes for building applications.	It is about building cars, machines, hardware, buildings, etc.
In Software Engineering Process construction and development cost is low.	In Conventional Engineering Process construction and development cost is high.
It can involve the application of new and untested elements in software projects.	It usually applies only known and tested principles to meet product requirements.
In Software Engineering Process, most development effort goes into building new designs and features.	In Conventional Engineering Process, most development efforts are required to change old designs.
Product development develops intangible products (software).	Product development develops tangible products (e.g. bridges, buildings).
Design requirements may change throughout the development process.	Design Requirements are typically well-defined upfront.
Testing is an integral part of the development process.	Testing occurs mainly after product completion.
Prototyping is common and helps to refine requirements.	Prototyping is less common due to cost and time.
Maintenance and updates are necessary to keep software relevant.	Maintenance is typically scheduled or reactive.
Software development often involves complex logic and algorithms.	Conventional engineering may have more complex physical properties to deal with.

SDLC – Software Development Life Cycle

What is SDLC?

The software development lifecycle (SDLC) is the cost-effective and time-efficient process that development teams use to design and build high-quality software. The goal of SDLC is to minimize project risks through forward planning so that software meets customer expectations during production and beyond. This methodology outlines a series of steps that divide the software development process into tasks you can assign, complete, and measure.

SDLC – Software Development Life Cycle

Why is SDLC important?

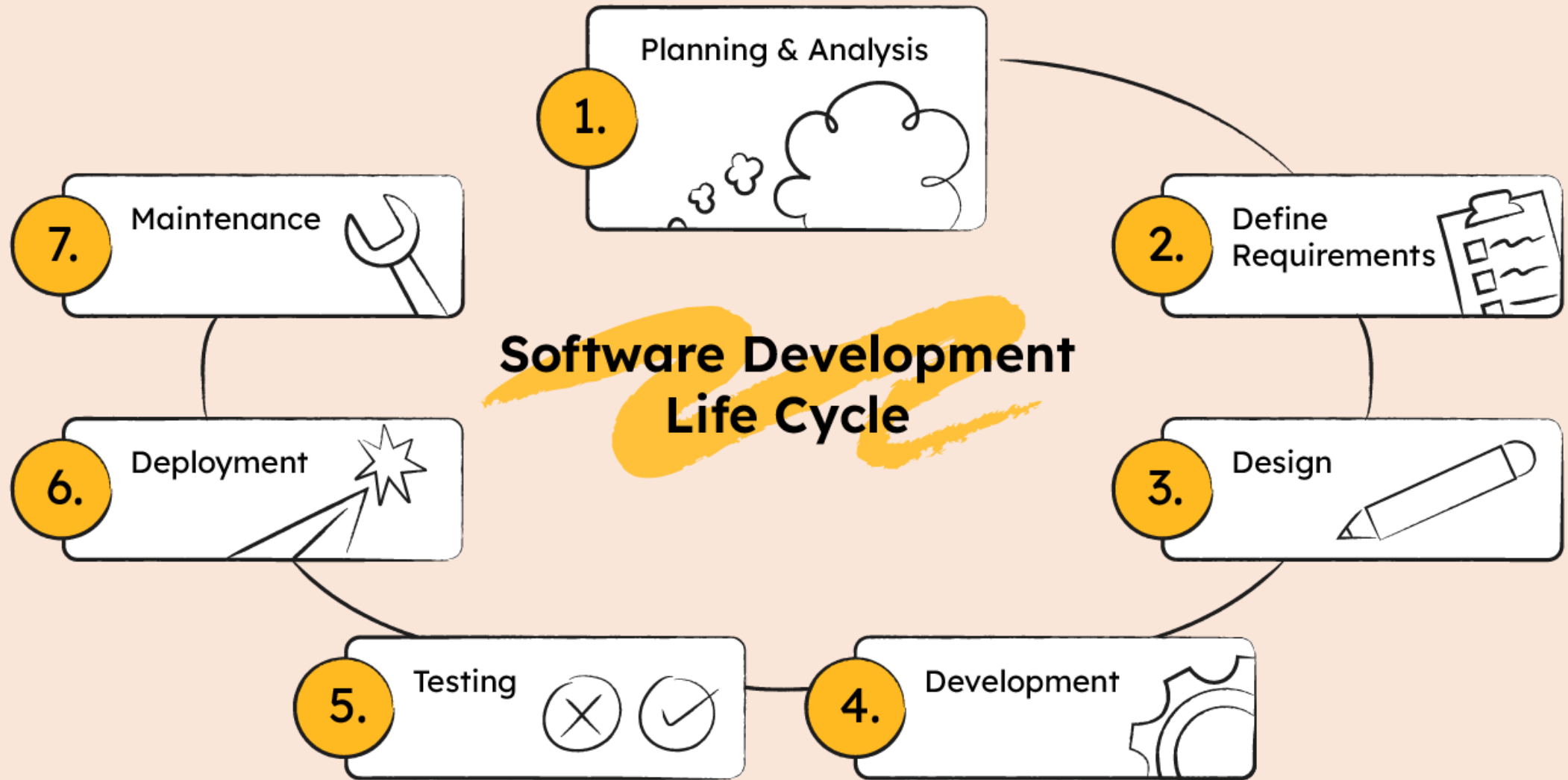
Software development can be challenging to manage due to changing requirements, technology upgrades, and cross-functional collaboration. The software development lifecycle (SDLC) methodology provides a systematic management framework with specific deliverables at every stage of the software development process. As a result, all stakeholders agree on software development goals and requirements upfront and also have a plan to achieve those goals.

SDLC – Software Development Life Cycle

Here are some benefits of SDLC:

- Increased visibility of the development process for all stakeholders involved
- Efficient estimation, planning, and scheduling
- Improved risk management and cost estimation
- Systematic software delivery and better customer satisfaction

Software Development Life Cycle Phases



Software Development Life Cycle Phases

Planning & Analysis

The first phase of the SDLC is the project planning stage where you are gathering business requirements from your client or stakeholders. This phase is when you evaluate the feasibility of creating the product, revenue potential, the cost of production, the needs of the end-users, etc.

Once it is decided that the software project is in line with business and stakeholder goals, feasible to create, and addresses user needs, then you can move on to the next phase.

Software Development Life Cycle Phases

Define Requirements

This phase is critical for converting the information gathered during the planning and analysis phase into clear requirements for the development team. This process guides the development of several important documents: a software requirement specification (SRS), a Use Case document, and a Requirement Traceability Matrix document.

Software Development Life Cycle Phases

Design

In the design phase, software engineers analyze requirements and identify the best solutions to create the software. For example, they may consider integrating pre-existing modules, make technology choices, and identify development tools. They will look at how to best integrate the new software into any existing IT infrastructure the organization may have.

Software Development Life Cycle Phases

Implement/Development

In the implementation phase, the development team codes the product. They analyze the requirements to identify smaller coding tasks they can do daily to achieve the final result.

Software Development Life Cycle Phases

Test

The development team combines automation and manual testing to check the software for bugs. Quality analysis includes testing the software for errors and checking if it meets customer requirements. Because many teams immediately test the code they write, the testing phase often runs parallel to the development phase.

Software Development Life Cycle Phases

Deploy

During the deployment phase, your final product is delivered to your intended user. You can automate this process and schedule your deployment depending on the type.

Software Development Life Cycle Phases

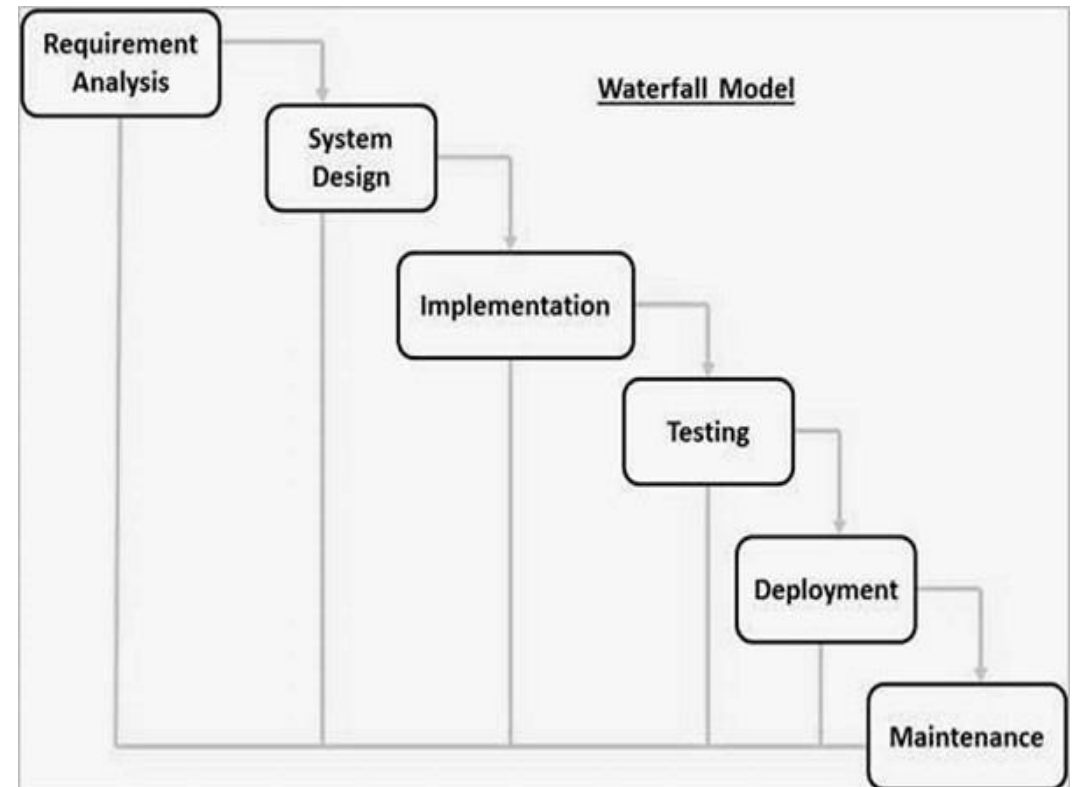
Maintenance

In the maintenance phase, among other tasks, the team fixes bugs, resolves customer issues, and manages software changes. In addition, the team monitors overall system performance, security, and user experience to identify new ways to improve the existing software.

Waterfall Model

Waterfall approach was first SDLC Model to be used widely in Software Engineering to ensure success of the project. In "The Waterfall" approach, the whole process of software development is divided into separate phases. In this Waterfall model, typically, the outcome of one phase acts as the input for the next phase sequentially.

All these phases are cascaded to each other in which progress is seen as flowing steadily downwards (like a waterfall) through the phases. The next phase is started only after the defined set of goals are achieved for previous phase and it is signed off, so the name "Waterfall Model". In this model, phases do not overlap.



Waterfall Model

Waterfall Model - Application

Every software developed is different and requires a suitable SDLC approach to be followed based on the internal and external factors. Some situations where the use of Waterfall model is most appropriate are –

- Requirements are very well documented, clear and fixed.
- Product definition is stable.
- Technology is understood and is not dynamic.
- There are no ambiguous requirements.
- Ample resources with required expertise are available to support the product.
- The project is short.

Waterfall Model - Advantages

- Simple and easy to understand and use
- Easy to manage due to the rigidity of the model. Each phase has specific deliverables and a review process.
- Phases are processed and completed one at a time.
- Works well for smaller projects where requirements are very well understood.
- Clearly defined stages.
- Well understood milestones.
- Easy to arrange tasks.
- Process and results are well documented.

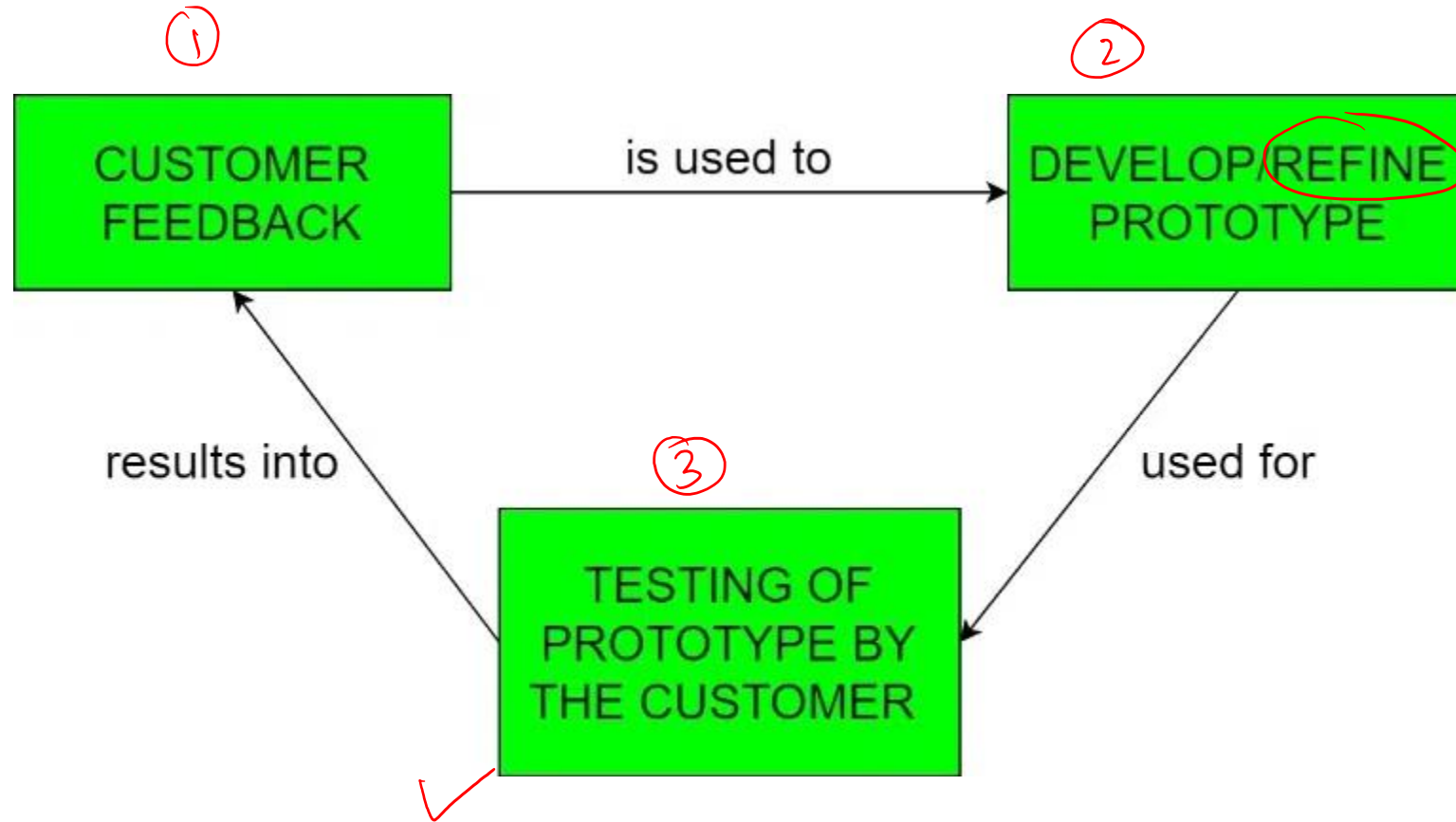
Waterfall Model - Disadvantages

- No working software is produced until late during the life cycle.
- High amounts of risk and uncertainty.
- Not a good model for complex and object-oriented projects.
- Poor model for long and ongoing projects.
- Not suitable for the projects where requirements are at a moderate to high risk of changing. So, risk and uncertainty is high with this process model.
- It is difficult to measure progress within stages.
- Cannot accommodate changing requirements.
- Adjusting scope during the life cycle can end a project.

Prototyping Model

- Prototyping is defined as the process of developing a working replication of a product or system that has to be engineered. It offers a small-scale facsimile of the end product and is used for obtaining customer feedback. The Prototyping concept is described below:
- The Prototyping Model is one of the most popularly used Software Development Life Cycle Models (SDLC models). This model is used when the customers do not know the exact project requirements beforehand. In this model, a prototype of the end product is first developed, tested, and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.

Prototyping Model



Prototyping Model

In this process model, the system is partially implemented before or during the analysis phase thereby giving the customers an opportunity to see the product early in the life cycle. The process starts by interviewing the customers and developing the incomplete high-level paper model. This document is used to build the initial prototype supporting only the basic functionality as desired by the customer. Once the customer figures out the problems, the prototype is further refined to eliminate them. The process continues until the user approves the prototype and finds the working model to be satisfactory.

Steps of Prototyping Model

Step 1: Requirement Gathering and Analysis: This is the initial step in designing a prototype model. In this phase, users are asked about what they expect or what they want from the system.

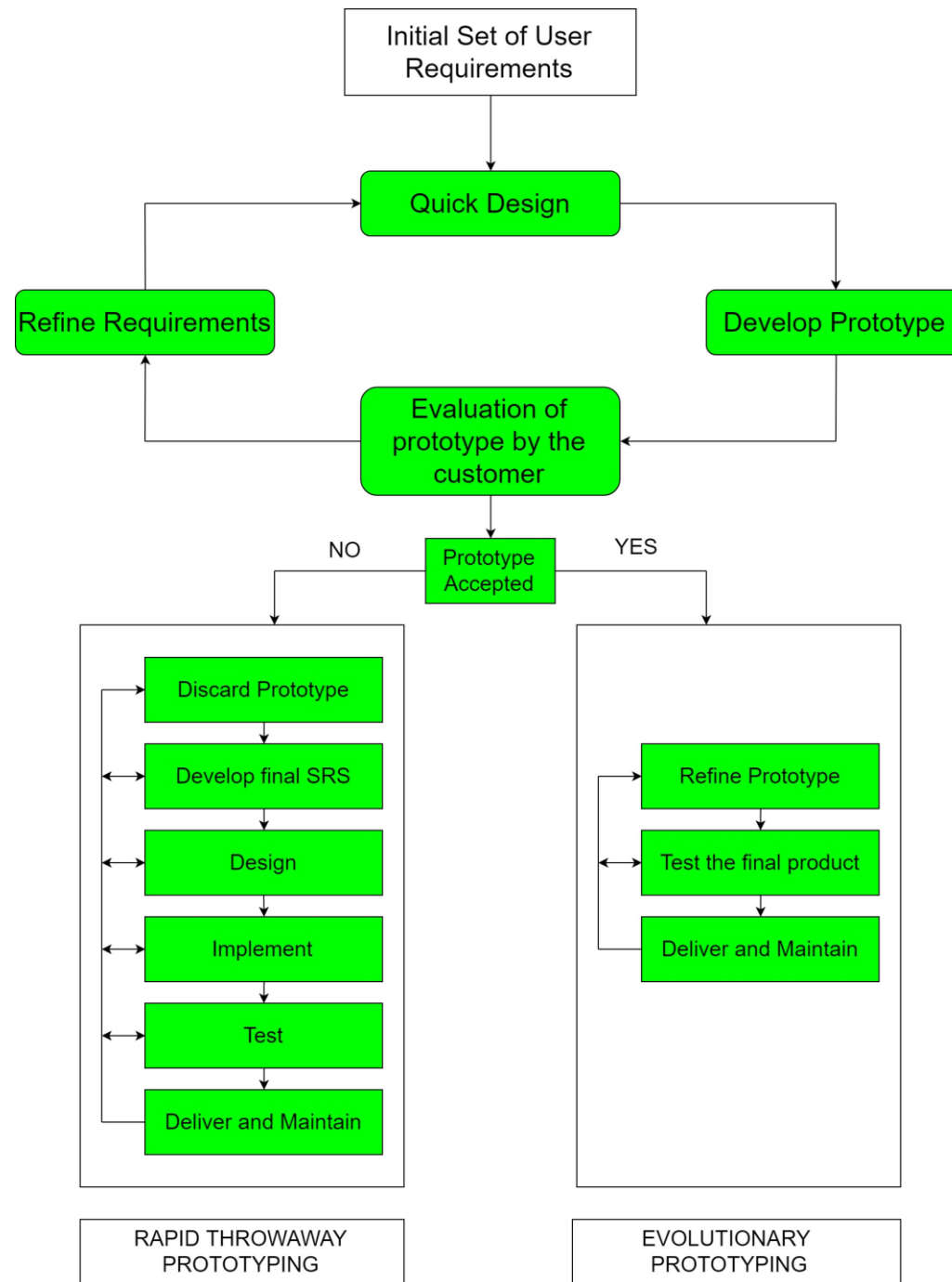
Step 2: Quick Design: This is the second step in Prototyping Model. This model covers the basic design of the requirement through which a quick overview can be easily described.

Step 3: Build a Prototype: This step helps in building an actual prototype from the knowledge gained from prototype design.

Step 4: Initial User Evaluation: This step describes the preliminary testing where the investigation of the performance model occurs, as the customer will tell the strength and weaknesses of the design, which was sent to the developer.

Step 5: Refining Prototype: If any feedback is given by the user, then improving the client's response to feedback and suggestions, the final system is approved.

Step 6: Implement Product and Maintain: This is the final step in the phase of the Prototyping Model where the final system is tested and distributed to production, here the program is run regularly to prevent failures.



Types of Prototyping Models

There are four types of Prototyping Models, which are described below:

- 1) Rapid Throwaway Prototyping - In this method, a developed prototype need not necessarily be a part of the ultimately accepted prototype.
- 2) Evolutionary Prototyping - the prototype developed initially is incrementally refined on the basis of customer feedback till it finally gets accepted. In comparison to Rapid Throwaway Prototyping, it offers a better approach that saves time as well as effort.
- 3) Incremental Prototyping - the final expected product is broken into different small pieces of prototypes and developed individually. In the end, when all individual pieces are properly developed, then the different prototypes are collectively merged into a single final product in their predefined order.
- 4) Extreme Prototyping - This method is mainly used for web development.

Advantages of Prototyping Model

- The customers get to see the partial product early in the life cycle. This ensures a greater level of customer satisfaction and comfort.
- New requirements can be easily accommodated as there is scope for refinement.
- Missing functionalities can be easily figured out.
- Errors can be detected much earlier thereby saving a lot of effort and cost, besides enhancing the quality of the software.
- The developed prototype can be reused by the developer for more complicated projects in the future.
- Flexibility in design.
- Early feedback from customers and stakeholders can help guide the development process and ensure that the final product meets their needs and expectations.
- Prototyping can be used to test and validate design decisions, allowing for adjustments to be made before significant resources are invested in development.
- Prototyping can help reduce the risk of project failure by identifying potential issues and addressing them early in the process.
- Prototyping can facilitate communication and collaboration among team members and stakeholders, improving overall project efficiency and effectiveness.
- Prototyping can help bridge the gap between technical and non-technical stakeholders by providing a tangible representation of the product.

Disadvantages of the Prototyping Model

- Costly with respect to time as well as money.
- There may be too much variation in requirements each time the prototype is evaluated by the customer.
- Poor Documentation due to continuously changing customer requirements.
- It is very difficult for developers to accommodate all the changes demanded by the customer.
- There is uncertainty in determining the number of iterations that would be required before the prototype is finally accepted by the customer.
- After seeing an early prototype, the customers sometimes demand the actual product to be delivered soon.
- Developers in a hurry to build prototypes may end up with sub-optimal solutions.
- The customer might lose interest in the product if he/she is not satisfied with the initial prototype.
- The prototype may not be scalable to meet the future needs of the customer.
- The prototype may not accurately represent the final product due to limited functionality or incomplete features.
- The focus on prototype development may shift the focus away from the final product, leading to delays in the development process.
- The prototype may give a false sense of completion, leading to the premature release of the product.
- The prototype may not consider technical feasibility and scalability issues that can arise during the final product development.
- The prototype may be developed using different tools and technologies, leading to additional training and maintenance costs.
- The prototype may not reflect the actual business requirements of the customer, leading to dissatisfaction with the final product.

Applications of Prototyping Model

- The Prototyping Model should be used when the requirements of the product are not clearly understood or are unstable.
- The prototyping model can also be used if requirements are changing quickly.
- This model can be successfully used for developing user interfaces, high-technology software-intensive systems, and systems with complex algorithms and interfaces.
- The prototyping Model is also a very good choice to demonstrate the technical feasibility of the product.

Spiral Model

The Spiral Model is a **Software Development Life Cycle (SDLC)** model that provides a systematic and iterative approach to software development. In its diagrammatic representation, looks like a spiral with many loops. The exact number of loops of the spiral is unknown and can vary from project to project. Each loop of the spiral is called a **Phase of the** software development process.

1. The exact number of phases needed to develop the product can be varied by the project manager depending upon the project risks.
2. As the project manager dynamically determines the number of phases, the project manager has an important role in developing a product using the spiral model.
3. It is based on the idea of a spiral, with each iteration of the spiral representing a complete software development cycle, from requirements gathering and analysis to design, implementation, testing, and maintenance.

Spiral Model - Phases

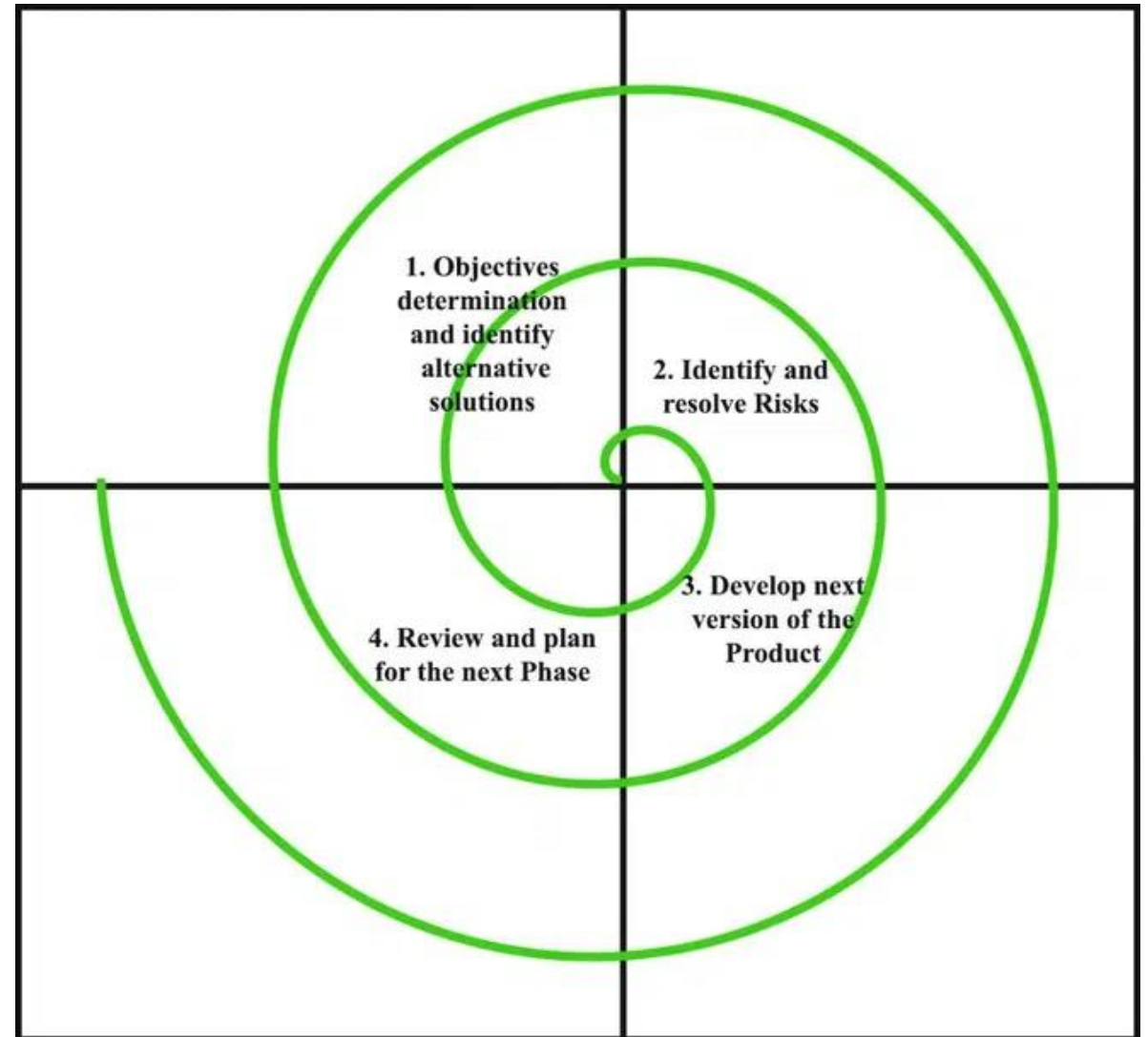
Each phase of the Spiral Model is divided into four quadrants as shown in the above figure. The functions of these four quadrants are discussed below:

1.Objectives determination and identify alternative solutions: Requirements are gathered from the customers and the objectives are identified, elaborated, and analyzed at the start of every phase. Then alternative solutions possible for the phase are proposed in this quadrant.

2.Identify and resolve Risks: During the second quadrant, all the possible solutions are evaluated to select the best possible solution. Then the risks associated with that solution are identified and the risks are resolved using the best possible strategy. At the end of this quadrant, the Prototype is built for the best possible solution.

3.Develop the next version of the Product: During the third quadrant, the identified features are developed and verified through testing. At the end of the third quadrant, the next version of the software is available.

4.Review and plan for the next Phase: In the fourth quadrant, the Customers evaluate the so-far developed version of the software. In the end, planning for the next phase is started.



Project Risks

1. **Cost risk** - Can be financial or time-based. A risk could be due to a budget that is too tight or the project taking too long to complete.
2. **Schedule risk** - An important factor that affects the project's success.
3. **Performance risk** - The risk that the project will fail to produce results consistent with project specifications.
4. **Resource risk** - A project can't run without its resources, and resources have a significant impact on a project's performance and pace. Common risks include staff shortages, staff skills, equipment unavailability, cost overruns, and delays.
5. **Scope creep** - One of the most common risks associated with projects. It usually occurs when the project deliverables are changed after they have already been defined.
6. **Financial risk** - Associated with a project's funding. A common financial risk is running out of money before project objectives are met

Risk Handling in Spiral Model

A risk is any adverse situation that might affect the successful completion of a software project. The most important feature of the spiral model is handling these unknown risks after the project has started. Such risk resolutions are easier done by developing a prototype.

1. The spiral model supports coping with risks by providing the scope to build a prototype at every phase of software development.
2. The Prototyping Model also supports risk handling, but the risks must be identified completely before the start of the development work of the project.
3. But in real life, project risk may occur after the development work starts, in that case, we cannot use the Prototyping Model.
4. In each phase of the Spiral Model, the features of the product are defined and analyzed, and the risks at that point in time are identified and are resolved through prototyping.
5. Thus, this model is much more flexible compared to other SDLC models.

Advantages of the Spiral Model

Risk Handling: The projects with many unknown risks that occur as the development proceeds, in that case, Spiral Model is the best development model to follow due to the risk analysis and risk handling at every phase.

Good for large projects: It is recommended to use the Spiral Model in large and complex projects.

Flexibility in Requirements: Change requests in the Requirements at a later phase can be incorporated accurately by using this model.

Customer Satisfaction: Customers can see the development of the product at the early phase of the software development and thus, they habituated with the system by using it before completion of the total product.

Iterative and Incremental Approach: The Spiral Model provides an iterative and incremental approach to software development, allowing for flexibility and adaptability in response to changing requirements or unexpected events.

Emphasis on Risk Management: The Spiral Model places a strong emphasis on risk management, which helps to minimize the impact of uncertainty and risk on the software development process.

Improved Communication: The Spiral Model provides for regular evaluations and reviews, which can improve communication between the customer and the development team.

Improved Quality: The Spiral Model allows for multiple iterations of the software development process, which can result in improved software quality and reliability

Disadvantages of the Spiral Model

Complex: The Spiral Model is much more complex than other SDLC models.

Expensive: Spiral Model is not suitable for small projects as it is expensive.

Too much dependability on Risk Analysis: The successful completion of the project is very much dependent on Risk Analysis. Without very highly experienced experts, it is going to be a failure to develop a project using this model.

Difficulty in time management: As the number of phases is unknown at the start of the project, time estimation is very difficult.

Complexity: The Spiral Model can be complex, as it involves multiple iterations of the software development process.

Time-Consuming: The Spiral Model can be time-consuming, as it requires multiple evaluations and reviews.

Resource Intensive: The Spiral Model can be resource-intensive, as it requires a significant investment in planning, risk analysis, and evaluations.

When to use Spiral Model?

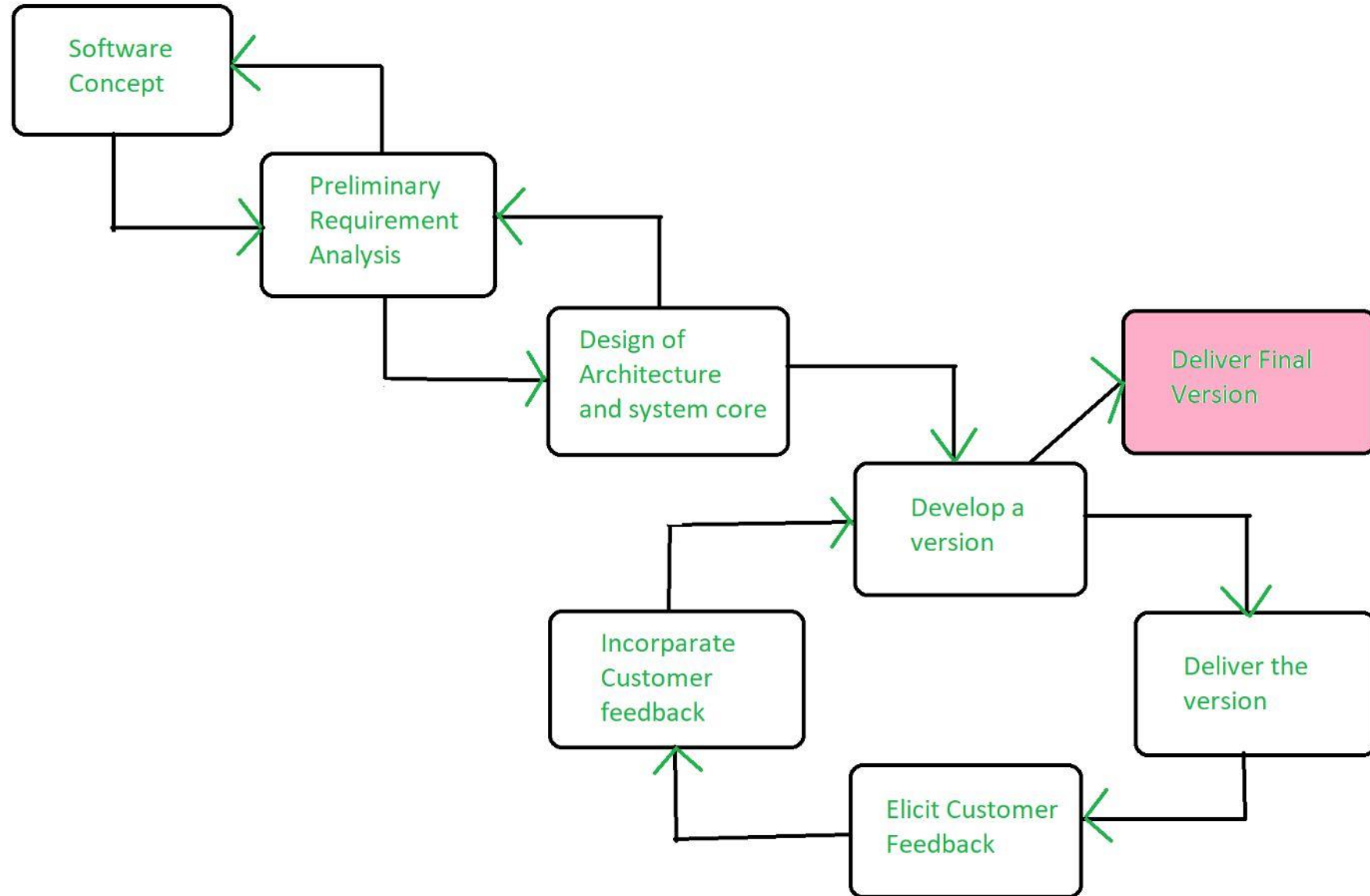
- When a project is vast in software engineering, a spiral model is utilized.
- A spiral approach is utilized when frequent releases are necessary.
- When it is appropriate to create a prototype
- When evaluating risks and costs is crucial
- The spiral approach is beneficial for projects with moderate to high risk.
- The SDLC's spiral model is helpful when requirements are complicated and ambiguous.
- If modifications are possible at any moment

Evolutionary Model

The Evolutionary development model divides the development cycle into smaller, incremental waterfall models in which users can get access to the product at the end of each cycle.

1. Feedback is provided by the users on the product for the planning stage of the next cycle and the development team responds, often by changing the product, plan, or process.
2. Therefore, the software product evolves with time.
3. All the models have the disadvantage that the duration of time from the start of the project to the delivery time of a solution is very high.
4. The evolutionary model solves this problem with a different approach.
5. The evolutionary model suggests breaking down work into smaller chunks, prioritizing them, and then delivering those chunks to the customer one by one.
6. The number of chunks is huge and is the number of deliveries made to the customer.
7. The main advantage is that the customer's confidence increases as he constantly gets quantifiable goods or services from the beginning of the project to verify and validate his requirements.
8. The model allows for changing requirements as well as all work is broken down into maintainable work chunks.

Evolutionary Model



Applications of Evolutionary Model

1. It is used in large projects where you can easily find modules for incremental implementation. Evolutionary model is commonly used when the customer wants to start using the core features instead of waiting for the full software.
2. Evolutionary model is also used in object oriented software development because the system can be easily portioned into units in terms of objects.

Necessary Conditions for Implementing this Model

1. Customer needs are clear and been explained in deep to the developer team.
2. There might be small changes required in separate parts but not a major change.
3. As it requires time, so there must be some time left for the market constraints.
4. Risk is high and continuous targets to achieve and report to customer repeatedly.
5. It is used when working on a technology is new and requires time to learn.

Advantages Evolutionary Model

- 1. Adaptability to Changing Requirements:** Evolutionary models work effectively in projects when the requirements are ambiguous or change often. They support adjustments and flexibility along the course of development.
- 2. Early and Gradual Distribution:** Functional components or prototypes can be delivered early thanks to incremental development. Faster user satisfaction and feedback may result from this.
- 3. User Commentary and Involvement:** Evolutionary models place a strong emphasis on ongoing user input and participation. This guarantees that the software offered closely matches the needs and expectations of the user.
- 4. Improved Handling of Difficult Projects:** Big, complex tasks can be effectively managed with the help of evolutionary models. The development process is made simpler by segmenting the project into smaller, easier-to-manage portions.

Disadvantages Evolutionary Model

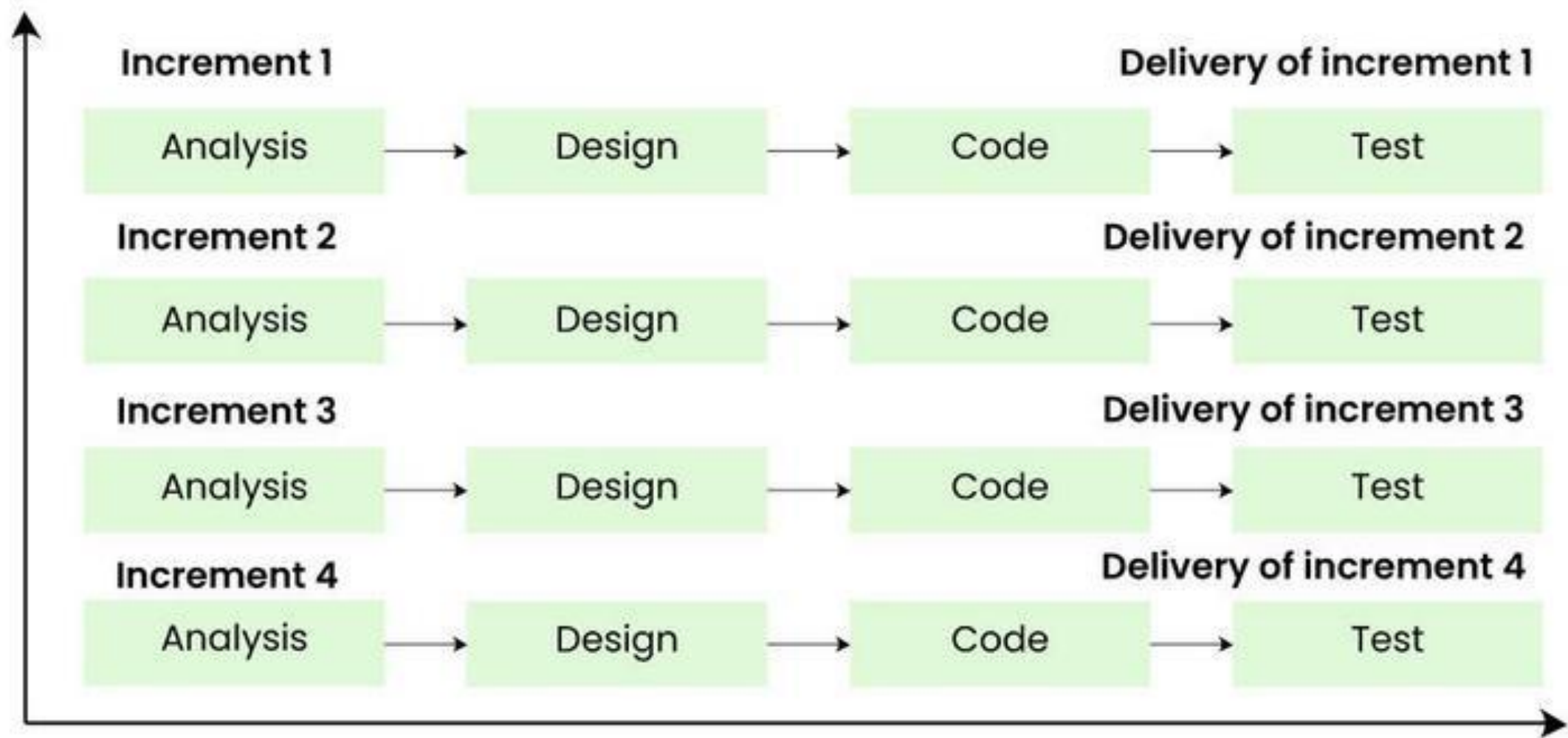
- 1. Communication Difficulties:** Evolutionary models require constant cooperation and communication. The strategy may be less effective if there are gaps in communication or if team members are spread out geographically.
- 2. Dependence on an Expert Group:** A knowledgeable and experienced group that can quickly adjust to changes is needed for evolutionary models. Teams lacking experience may find it difficult to handle these model's dynamic nature.
- 3. Increasing Management Complexity:** Complexity can be introduced by organizing and managing several increments or iterations, particularly in large projects. In order to guarantee integration and synchronization, good project management is needed.
- 4. Greater Initial Expenditure:** As evolutionary models necessitate continual testing, user feedback and prototyping, they may come with a greater starting cost. This may be a problem for projects that have limited funding.

Iterative Enhancement Model

In software development, the Iterative Enhancement Model stands out due to its incremental and iterative nature, it is also known as an incremental model. This approach focuses on incremental development and improvement rather than trying to complete a software product in a single phase. This model is based on segmenting the project into smaller units, or iterations, with a set of tasks completed.

Software development uses a dynamic and adaptable method called the iterative enhancement Model. The iterative enhancement model encourages a software product's ongoing evolution and improvement. This methodology is noticeable due to its concentration on adaptability, flexibility and change responsiveness. It makes it easier for a product to evolve because it gives developers the freedom to progressively enhance the software, making sure that it complies with evolving specifications, user demands, and market demands. This helps products evolve more easily.

Iterative Enhancement Model



Iterative Enhancement Model

The Iterative Enhancement Model creates an environment where development teams can more effectively adjust to changing requirements by segmenting the software development process into smaller, more manageable parts. Every iteration improves on the one before it, adding new features and fixing problems found in earlier stages. Members of the team, stakeholders and end users are encouraged to collaborate and communicate continuously to make sure the software meets changing needs and expectations. Until the software is finished being built, the iteration process is carried out, which involves giving the user the increments.

Advantages of Iterative Enhancement Model

1. Adaptation to changing requirements is made possible by its flexibility in accomodating modifications and improvement during each iteration.
2. Early software iterations provide clients with functional portions of the product, facilitating prompt feedback and validation.
3. Problems and risks can be identified and addressed early in the developement process, reduces chances of issue for future stages.
4. Feedback and constant client involvement are encouraged to make sure the finished product lives up to user expectations.
5. Every iteration is put through testing and improvement, leading to higher quality product.

Disadvantages of Iterative Enhancement Model

1. Especially in larger projects, managing several iterations at once can add complexity.
2. Higher cost
3. Due to constant changes, there may be delays in documentation, making it more difficult to maintain comprehensive documentation.
4. Continuous customer engagement may not be possible in all scenarios, which impacts the effectiveness of the model.

Specific use cases where this model proves beneficial include:

1. **Mobile app development:** Updates and improvements are often needed for mobile apps to stay current with new devices, operating system versions and user preferences. By using an iterative process developers can release the beta versions of their apps, get user feedback and then improve functionality of those iterations in future release.
2. **Web Application Development:** The requirements for developing web applications frequently change as a result of shifting user demand and advancements in technology. The Iterative Enhancement Model makes it possible to developed features incrementally and guaranteeing that the application can be modified to satisfy changing user and market demands. In later iterations it also makes it easier to incorporate new features based on input from users.
3. **E-commerce Platforms:** Developement in e-commerece field often involves constant updates. Implementing an iterative approach enables the introduction of new functionality.

